Guía de defensa

Maqueta de inicio de sesión, pacientes y aprobación · Dientecito Feliz · Clínica Odontológica UAM

1. Qué es lo que entrego

Una aplicación web hecha en **Python con Flask** que cubre el inicio de sesión, el registro de pacientes por el estudiante, la aprobación de esos pacientes por el profesor, y la descarga del expediente clínico — esta última solo para el profesor, el estudiante no la ve. Todo se guarda en listas y diccionarios de Python en memoria: **todavía no hay SQL ni SQLite**, los datos se pierden al reiniciar el servidor. Las demás secciones (Citas, Salas, Tarifario, Usuarios, Bitácora) siguen sin desarrollar y se muestran marcadas «En proceso». Toda la lógica está en Python: ninguna página lleva JavaScript.

Archivos: login.py (136 líneas, toda la lógica), templates/entrar.html (91 líneas, la pantalla de entrada), templates/base.html (29 líneas, cabecera y pestañas que comparten las demás pantallas), templates/menu.html (17 líneas, el inicio) y templates/pacientes.html (59 líneas, el registro y la lista de pacientes), más static/ con las imágenes y la hoja de estilos del proyecto.

Para ejecutarlo: python login.py y abrir http://localhost:5001.

2. El recorrido completo

Inicio de sesión (igual que la entrega anterior)

1. La persona abre http://localhost:5001 y entra con su CIF y contraseña; entrar() revisa los datos contra CUENTAS.
2. Si están bien, guarda el CIF y el rol en la sesión y redirige a /menu, que muestra la pestaña «Inicio».

Registrar y aprobar un paciente (lo nuevo de esta entrega)

1. El estudiante entra a la pestaña «Mis pacientes» (ruta /pacientes) y llena el formulario: nombre, cédula, teléfono y motivo de consulta.
2. La función pacientes() revisa que el nombre y la cédula no estén vacíos y agrega el paciente a la lista PACIENTES con aprobado=False.
3. El profesor entra a su propia pestaña «Pacientes» y ve a todos los pacientes de todos los estudiantes, con su estado (Pendiente o Aprobado).
4. El profesor presiona «Aprobar»: un formulario envía por POST el id del paciente y aprobado=1; pacientes() lo busca en la lista y cambia su estado.
5. Solo el profesor ve el enlace «Descargar expediente». Lleva a /pacientes/<id>/expediente.txt, que arma un archivo de texto con los datos del paciente y lo manda como descarga. Si un estudiante entra a esa dirección a mano, la ruta lo manda de vuelta al menú sin mostrarle nada.

3. login.py, línea por línea

Es el archivo por el que más van a preguntar: aquí está todo lo que decide, con énfasis en lo que se agregó en esta entrega.

	Código	Qué decir si el profesor pregunta
1-6	`"""Maqueta de Dientecito Feliz: entrada, pacientes..."""`	El docstring del archivo: explica qué hace el programa y cómo se ejecuta. Ahora menciona que no hay SQL todavía.
7	`import itertools`	De la librería estándar: itertools.count() arma un contador que nunca se repite, para darle un id a cada paciente nuevo.
9	`from flask import Flask, Response, redirect, ...`	Se agregó Response frente a la entrega anterior: lo uso para armar la respuesta de texto que se descarga en /expediente.txt, con sus propios encabezados.
12-13	`CUENTAS = {...} NOMBRES = {...}`	CUENTAS ya existía. NOMBRES es nuevo: el nombre que se muestra en la columna «Estudiante» cuando el profesor ve la lista de pacientes.
17-19	`MENUS = {"profesor": [("Inicio", "menu"), ("Pacientes", "pacientes"), ...]`	Antes cada pestaña era solo un texto; ahora es una pareja (nombre, endpoint). Si el endpoint es None la pestaña sigue apuntando a # («en proceso»); «Pacientes» y «Mis pacientes» ya apuntan a la función pacientes().
20-24	`SECCIONES = {...}`	Ya no incluye «Pacientes» ni «Mis pacientes»: esa sección dejó de estar «en proceso», así que se quitó de la tabla de pendientes del inicio.
30-31	`PACIENTES = [] _siguiente_id = itertools.count(1)`	Aquí vive toda la información de esta entrega: una lista de diccionarios, uno por paciente, con un contador aparte para los ids. No es una base de datos: es una lista en la memoria del proceso de Python, se vacía cada vez que se reinicia el servidor.
72-73	`@app.route("/pacientes", methods=["GET", "POST"]) def pacientes():`	Una sola función atiende tanto el formulario de registro como la tabla y el botón de aprobar; decide qué hacer según el campo accion que vino en el POST.
81-92	`if ... accion == "registrar" and rol == "estudiante":`	Solo entra aquí si quien lo envía es estudiante: aunque alguien arme el POST a mano, el profesor nunca puede registrar un paciente por esta vía. Valida nombre y cédula igual que el formulario de entrar(), y agrega el diccionario a PACIENTES con aprobado en False.
94-100	`elif ... accion == "aprobar" and rol == "profesor":`	Lo mismo pero al revés: solo el profesor puede aprobar. Busca el paciente por id con un for y le cambia el campo aprobado; no hace falta una base de datos para un UPDATE tan chico.
102	`datos["pacientes"] = PACIENTES if rol == "profesor" else [...]`	Aquí está el filtro por rol: el profesor ve la lista completa, el estudiante solo los pacientes cuyo estudiante_cif es el suyo. Es la misma idea de «alcance» que usa el sistema completo con SQL, pero con una comprensión de lista.
106-124	`@app.route("/pacientes/<int:pid> /expediente.txt") def expediente_txt(pid):`	La ruta de descarga. Lo primero que hace, antes de tocar ningún dato, es revisar que session.get("rol") sea "profesor"; si no lo es (incluido el caso de no haber entrado) lo redirige y nunca arma el archivo. Es la respuesta a «que el profesor pueda descargar el expediente pero el estudiante no».
114-122	`contenido = ( "Expediente clínico..." f"Nombre: ..."`	Arma el texto del expediente con un f-string de varias líneas. Es texto plano por ahora, no un PDF: para esta maqueta alcanza con mostrar que la descarga está restringida por rol; el formato final queda para una entrega posterior.

	Código	Qué decir si el profesor pregunta
123-124	`return Response(contenido, mimetype="text/plain", headers={"Content-Disposition": ...})`	Response arma la respuesta a mano en vez de usar render_template: le doy el texto, el tipo de contenido y la cabecera Content-Disposition con attachment, que es lo que le dice al navegador «esto se descarga», no «esto se muestra».

4. templates/entrar.html

No cambió frente a la entrega anterior: sigue siendo la pantalla de inicio de sesión, con su propio <style> y sin JavaScript. Ver la guía de la entrega anterior para el detalle línea por línea si el profesor pregunta por ella.

5. base.html, menu.html y pacientes.html

base.html es nuevo: junta la cabecera («Dientecito Feliz», el CIF de quien entró, «Salir») y la barra de pestañas en un solo archivo, con dos bloques de Jinja ({% block titulo %} y {% block contenido %}) que cada pantalla llena a su manera con {% extends "base.html" %}. Antes esa cabecera estaba repetida dentro de menu.html; con dos pantallas reales (inicio y pacientes) convenía tenerla en un solo lugar.

La barra de pestañas ahora arma cada enlace con `url_for(endpoint) if endpoint else '#'`: las pestañas resueltas (Inicio, Pacientes) llevan a su página; las que siguen «en proceso» apuntan a # igual que antes.

pacientes.html muestra el formulario de registro solo si rol == "Estudiante", y en la tabla la columna «Estudiante», el botón de aprobar y el enlace de descarga solo si rol == "Profesor" — es la misma idea de la plantilla decidiendo qué mostrar según un valor que ya le mandó Python, igual que el {% if %} de entrar.html.

6. Preguntas probables y respuesta corta

¿Por qué no usan SQL o SQLite todavía?

Porque esta entrega se concentró en el flujo de registro y aprobación; PACIENTES es una lista en memoria a propósito, para no mezclar dos cosas nuevas a la vez. El sistema completo (app.py) ya usa SQLite; pasar la maqueta a una base de datos queda para una entrega posterior.

¿Qué pasa con los pacientes si reinicio el servidor?

Se pierden: viven en una lista de Python en la memoria del proceso, no en disco. Es la consecuencia directa de no tener base de datos todavía.

¿Cómo deciden quién puede aprobar y quién puede descargar?

Con el rol que quedó guardado en la sesión al entrar. pacientes() revisa rol == "profesor" antes de aprobar, y expediente_txt() hace lo mismo antes de armar el archivo; si la condición falla, ni siquiera llega a tocar los datos del paciente.

¿Qué pasa si un estudiante entra directo a /pacientes/1/expediente.txt?

Nada: expediente_txt() revisa session.get("rol") antes que cualquier otra cosa y lo redirige, sea porque no entró o porque entró como estudiante. No hay forma de llegar al archivo sin ser profesor.

¿Por qué el expediente se descarga en .txt y no en PDF?

Porque esta entrega prueba el control de acceso (quién puede descargar), no el formato final del documento. Cambiar Response por un PDF generado, por ejemplo con reportlab, no afecta esa lógica.

¿Por qué una sola función pacientes() atiende el registro y la aprobación?

Porque las dos acciones ocurren en la misma pantalla y dependen del mismo formulario; separarlas en dos funciones hubiera significado repetir la carga de la lista de pacientes y el armado de las pestañas.

¿Qué falta para que esto sea el sistema completo?

La base de datos, el rol de servicios generales, el resto de las secciones (Citas, Salas, Tarifario, Usuarios, Bitácora) y que la descarga del expediente sea un documento real en vez de texto plano.

7. Lo que esta entrega no hace (decirlo antes de que lo pregunten)

- No hay base de datos: los pacientes viven en una lista de Python que se vacía al reiniciar el servidor.
- El expediente se descarga en texto plano, no en PDF ni con el resto de las secciones de la historia clínica.
- No hay manera de rechazar o archivar un paciente, solo de aprobarlo o quitarle la aprobación.
- Solo hay dos roles de ejemplo, profesor y estudiante; falta servicios generales.
- Las demás pestañas (Citas, Salas, Tarifario, Usuarios, Bitácora) siguen sin desarrollar.
- No hay recuperación de contraseña ni ingreso con la cuenta institucional.

Decir esto de entrada demuestra que uno sabe dónde está parado y evita que la pregunta llegue como una objeción.